

Project Testing

ServiceNow Incident – Client Scripts and UI Policies

Purpose. This document briefly explains how to test a ServiceNow Incident configuration that uses UI Policies and Client Scripts to improve data quality and control user actions.

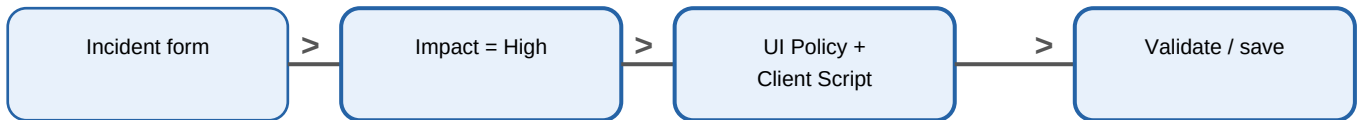


Figure 1. Testing flow from Incident entry to validation.

1. Project Overview

The project enforces consistent Incident data entry. When Impact is set to **High**, the configuration makes Assignment group mandatory, makes Urgency read-only, sets Urgency to High, and prevents saving when Assigned To is empty. Direct list editing of State is also blocked.

2. Testing Objectives

- Confirm that the UI Policy activates only for Impact = High.
- Verify that fields become mandatory or read-only as configured.
- Confirm that the onChange script automatically sets Urgency.
- Verify that the onSubmit script blocks invalid records.
- Confirm that onCellEdit prevents State changes from a list.
- Check that behavior reverses when Impact changes from High to another value.

3. Test Cases

ID	Test scenario	Expected result
TC-01	Set Impact to High	Assignment group becomes mandatory; Urgency becomes read-only.
TC-02	Change Impact to High	Urgency is automatically set to High and an information message appears.
TC-03	Save with High Impact and no Assigned To	Save is blocked and an error message is shown.
TC-04	Fill Assigned To and save	Incident saves successfully.
TC-05	Change Impact from High to Medium	Mandatory/read-only behavior is reversed.
TC-06	Edit State from Incident list	Alert appears and the State value does not change.
TC-07	Change State from the form	State change saves successfully.

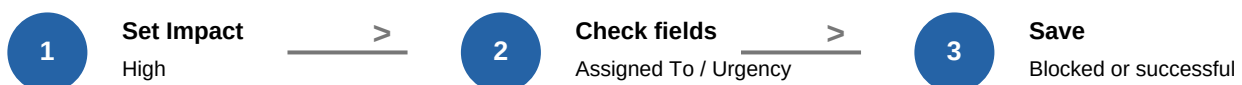


Figure 2. Example test sequence for a high-impact Incident.

4. Test Procedure

1. Open Incident → Create New.
2. Set Impact to 1 – High.
3. Check Assignment group, Urgency, and Assigned To behavior.
4. Attempt to save without Assigned To.
5. Complete Assigned To and save again.
6. Change Impact to Medium and confirm that the controls reverse.
7. Open the Incident list and try to edit State directly.
8. Open the record form, change State, and verify that the update is allowed.

5. Acceptance Criteria

- All test cases produce the expected result.
- Invalid high-impact Incidents cannot be submitted.
- Valid Incidents can be saved without unnecessary blocking.
- The configuration does not interfere with normal form-based updates.
- Users receive clear messages when an action is prevented.

6. Conclusion

Testing confirms that the UI Policy and Client Scripts work together to enforce Incident data quality. The configuration supports mandatory-field enforcement, automatic value updates, conditional read-only behavior, save validation, and protection against unauthorized list edits.